AcquirerX Deep-Dive 8/10 — Settlement Service

> **Service:** `settlement-service` **Port:** `9095` **Database:** `acquirerx_settlement` **Role:** End-of-day settlement, payouts to merchants, financial adjustments

---

1. What It Is

Settlement-service is the **money-mover**. After the merchant has processed transactions all day, this service:

1. **Aggregates** the day's approved transactions per merchant
2. **Calculates** gross amount, fees, net amount payable
3. **Creates** a SettlementBatch record per merchant per period
4. **Initiates** payouts to merchant bank accounts
5. **Posts** adjustments (reversals, chargebacks, manual corrections)

This service runs the **scheduled EOD job** that triggers settlement automatically every night, but it also exposes manual endpoints for ad-hoc processing.

---

2. Why It's a Separate Service

- **Scheduled, batch nature** — settlement runs in bursts (1 AM nightly), unlike real-time services
- **Heavy aggregations** — sum/group queries over millions of transactions
- **Banking integration boundary** — payouts will eventually integrate with NEFT/RTGS APIs; isolating that integration here keeps it from leaking into transaction processing
- **Compliance-heavy** — settlement records have stricter audit and retention requirements
- **Independent failure mode** — if settlement breaks, transactions still flow; only the next-day payouts are delayed

---

3. Tech Stack

- Spring Boot 3.x
- Spring Data JPA + MySQL

- Spring `@Scheduled` for the EOD cron job
- Spring Cloud OpenFeign — calls merchant-service and transaction-service
- Resilience4J — circuit breakers on Feign
- Lombok

---

4. Code Structure

```
settlement-service/src/main/java/com/acquirerx/settlement/
├── SettlementServiceApplication.java
├── client/
│   ├── MerchantServiceClient.java + MerchantServiceClientFallback.java
│   └── TransactionServiceClient.java + TransactionServiceClientFallback.java
└── settlement/
    ├── controller/
    │   ├── SettlementController.java
    │   └── PayoutController.java
    ├── dto/
    ├── entity/
    │   ├── SettlementBatch.java
    │   ├── Payout.java
    │   └── Adjustment.java
    └── service/
        ├── SettlementService.java
        ├── SettlementScheduler.java
        └── PayoutService.java
```

Three services, three concerns:

- `SettlementService` — creating settlement batches, querying summaries
- `SettlementScheduler` — the @Scheduled trigger
- `PayoutService` — payout creation, sync and async modes

---

5. Entities

5.1 `SettlementBatch` (table: `settlement_batch`)

The aggregate of one merchant's transactions for one settlement period.

Field	Type	Purpose
`settleBatchId`	Long PK	
`merchantId`	Long FK	
`grossAmount`	Double	Total of all txn amounts
`totalFees`	Double	Total of all fees collected
`netAmount`	Double	grossAmount − totalFees (= what merchant gets)
`txnCount`	Integer	How many transactions in this batch
`status`	String	OPEN → POSTED → PAID
`periodStart`	LocalDateTime	First txn time included
`periodEnd`	LocalDateTime	Last txn time included
`postedDate`	LocalDateTime	When the batch was created

Status flow:

- `OPEN` — created but not finalized (rare, intermediate state)
- `POSTED` — finalized, ready for payout
- `PAID` — payout has been issued

Why store grossAmount, totalFees, AND netAmount (redundant?): Yes, technically `netAmount = grossAmount - totalFees`, so it's redundant. But storing all three:

- Reports query the field directly (no calculation overhead)
- If fee logic changes later, historical batches retain their actual computed values
- Auditors can verify each batch independently

5.2 `Payout` (table: `payout`)

The actual money movement instruction.

Field	Type	Purpose
`payoutId`	Long PK	
`settleBatchId`	Long FK	Which batch this payout settles
`merchantId`	Long FK	
`amount`	Double	Amount being paid
`bankAccountRef`	String	Masked bank reference
`status`	String	PENDING → IN_PROGRESS → COMPLETED / FAILED
`payoutDate`	LocalDateTime	

Why batch and payout are separate entities:

- Batch = what was settled (financial accounting)
- Payout = how it was paid (banking integration)
- One batch can have multiple payouts (e.g., partial payouts or re-attempts on failure)

5.3 `Adjustment` (table: `adjustment`)

Manual corrections or chargeback debits.

Field	Type	Purpose
`adjustmentId`	Long PK	
`merchantId`	Long FK	
`txnId`	Long FK	The transaction being adjusted (nullable for manual adjustments)
`amount`	Double	Can be positive or negative
`reason`	String	Required text explanation
`type`	String	CHARGEBACK / REVERSAL / CORRECTION / MANUAL
`status`	String	PENDING / POSTED
`postedDate`	LocalDateTime	

Why adjustments exist:

- Chargebacks come in days/weeks after the original transaction; the merchant has already been paid, so the chargeback debits the merchant's next settlement
- Operations may need to manually correct disputed amounts
- Refunds processed outside the standard refund flow need recording

---

6. The SettlementScheduler

You provided the full code. Walkthrough:

```java
```

```java
@Slf4j
@Component
@RequiredArgsConstructor
public class SettlementScheduler {

    private final SettlementService settlementService;
    private final MerchantServiceClient merchantClient;

    @Scheduled(cron = "0 0 1 * * *")
    public void runEodSettlement() {
        log.info("=== EOD Settlement Job Started ===");

        List<Map<String, Object>> merchants;
        try {
            Map<String, Object> resp = merchantClient.getActiveMerchants(0, 1000);
            Object dataObj = resp.get("data");

            if (dataObj instanceof Map) {
                Map<String, Object> pagedData = (Map<String, Object>) dataObj;
                merchants = (List<Map<String, Object>>) pagedData.get("content");
            } else if (dataObj instanceof List) {
                merchants = (List<Map<String, Object>>) dataObj;
            } else {
                log.warn("Unexpected response format from merchant-service");
                return;
            }
        } catch (Exception e) {
            log.error("Failed to fetch active merchants: {}", e.getMessage());
            return;
        }

        if (merchants == null || merchants.isEmpty()) {
            log.info("No active merchants found");
            return;
        }

        int success = 0;
        int skip = 0;
        int fail = 0;

        for (Map<String, Object> merchant : merchants) {
            Long merchantId = Long.valueOf(merchant.get("merchantId").toString());
            try {
```

```
                settlementService.settle(merchantId);
                success++;
            } catch (IllegalStateException e) {
                skip++;
            } catch (Exception e) {
                log.error("Settlement failed for merchant {}: {}", merchantId, e.get
                fail++;
            }
        }

        log.info("=== EOD Settlement Completed === success={}, skipped={}, failed={}
                success, skip, fail);
    }

    public void runManualSettlement() {
        log.info("Manual settlement triggered");
        runEodSettlement();
    }
}
```

6.1 The cron expression

`0 0 1 * * *` = "at second 0, minute 0, hour 1, every day, every month, every day-of-week" = **1:00 AM daily**

Why 1 AM:

- After the business day has ended (transactions stopped or slowed)
- Before banking systems open in the morning (so payouts can flow at start-of-day)
- Avoids midnight (when other system jobs often run)

6.2 Why fetch all merchants in one call (page size 1000)

```java
merchantClient.getActiveMerchants(0, 1000);
```

For a learning project with a few hundred merchants, fetching all in one paged call is fine. For 100,000+ merchants, you'd:

- Iterate pages
- Or partition by merchant ID range across multiple workers

- Or use a job queue (Kafka, RabbitMQ) where each merchant is a separate task

6.3 Defensive response parsing

```java
if (dataObj instanceof Map) {
    Map<String, Object> pagedData = (Map<String, Object>) dataObj;
    merchants = (List<Map<String, Object>>) pagedData.get("content");
} else if (dataObj instanceof List) {
    merchants = (List<Map<String, Object>>) dataObj;
}
```

The merchant-service might return either:

- A paged response: `{ data: { content: [...], page, ... } }`
- A direct list: `{ data: [...] }`

This handles both. Defensive coding because the response shape evolved over time and isn't strictly enforced via shared DTOs (cross-service).

6.4 The success/skip/fail accounting

```java
for (Map<String, Object> merchant : merchants) {
    try {
        settlementService.settle(merchantId);
        success++;
    } catch (IllegalStateException e) {
        skip++;        // expected: no unsettled txns
    } catch (Exception e) {
        log.error(...);
        fail++;        // unexpected: actual error
    }
}
```

Why distinguish skip from fail:

- `IllegalStateException` is expected — it's how `settle()` signals "no unsettled txns to process for this merchant" (perfectly valid, e.g., a merchant who didn't trade today)
- Other exceptions are unexpected errors — DB issue, downstream service down, etc.

This distinction prevents the operations team from getting alerts for normal "no transactions" cases.

6.5 The runManualSettlement hook

```java
public void runManualSettlement() {
    log.info("Manual settlement triggered");
    runEodSettlement();
}
```

Useful for:

- Testing in dev
- Disaster recovery: scheduler crashed, run it manually
- Re-running after a failed scheduled run

Not exposed as an HTTP endpoint by default but can be added.

6.6 Enabling @Scheduled

For `@Scheduled` to work, the application class must have:

```java
@SpringBootApplication
@EnableDiscoveryClient
@EnableScheduling     // ← critical
public class SettlementServiceApplication { ... }
```

Without `@EnableScheduling`, the cron simply doesn't fire.

---

7. SettlementService — the settle method

The core method called by the scheduler:

```java
```

```java
@Transactional
public SettlementBatchResponseDTO settle(Long merchantId) {
    // Fetch unsettled txns from transaction-service via Feign
    Map<String, Object> resp = transactionClient.getUnsettledTxns(merchantId);
    List<Map<String, Object>> unsettled = extractList(resp, "data");

    if (unsettled == null || unsettled.isEmpty()) {
        throw new IllegalStateException("No unsettled txns for merchant " + merchant
    }

    // Aggregate
    double gross = 0, fees = 0, net = 0;
    LocalDateTime first = null, last = null;
    for (Map<String, Object> txn : unsettled) {
        gross += toDouble(txn.get("amount"));
        fees += toDouble(txn.get("totalFee"));
        net += toDouble(txn.get("netMerchantAmount"));
        LocalDateTime time = parseTime(txn.get("txnDate"));
        if (first == null || time.isBefore(first)) first = time;
        if (last == null || time.isAfter(last)) last = time;
    }

    // Create batch
    SettlementBatch batch = new SettlementBatch();
    batch.setMerchantId(merchantId);
    batch.setGrossAmount(gross);
    batch.setTotalFees(fees);
    batch.setNetAmount(net);
    batch.setTxnCount(unsettled.size());
    batch.setStatus("POSTED");
    batch.setPeriodStart(first);
    batch.setPeriodEnd(last);
    batch.setPostedDate(LocalDateTime.now());
    SettlementBatch saved = batchRepository.save(batch);

    // Mark txns settled (Feign call)
    transactionClient.markTxnsSettled(merchantId);

    log.info("Settlement created: batchId={}, merchantId={}, txns={}, net={}",
             saved.getSettleBatchId(), merchantId, unsettled.size(), net);
```

```
    return toResponse(saved);
}
```

7.1 Why @Transactional

If anything fails between batch creation and `markTxnsSettled`, we want a rollback. Otherwise:

- Batch saved
- Mark-settled call fails
- Next run picks up the same txns again → duplicate batches → duplicate payouts

The transaction boundary keeps it atomic.

7.2 The two-step "create batch then mark settled" pattern

This is a **distributed transaction** problem. The batch is in `acquirerx_settlement`; the settled flag is on Txn rows in `acquirerx_transaction`. They're in different DBs.

@Transactional only covers the local DB. If the Feign call to `markTxnsSettled` fails AFTER the batch is committed, you'd have a batch with no marked txns.

Mitigations in current design:

- Try-catch around the Feign call with rollback / compensating action (delete batch on failure)
- Idempotency on `markTxnsSettled` so retries are safe

Production-grade alternative:

- Saga pattern with event-driven choreography
- Outbox pattern: write a "settlement_event" row in same transaction as batch, separate worker picks it up to mark settled

For a learning project, the simple try-catch is acceptable.

7.3 Other SettlementService methods

java

```java
public PagedResponseDTO<SettlementBatchResponseDTO> getAll(PaginationParams paginati
public PagedResponseDTO<SettlementBatchResponseDTO> search(SettlementFilterDTO filte
public PagedResponseDTO<SettlementBatchResponseDTO> getByMerchant(Long merchantId, .
public SettlementSummaryDTO getMerchantSummary(Long merchantId);
public SettlementStatsDTO getStats();
```

Standard list/search/filter operations.

---

8. PayoutService

java

```java
@Service
@RequiredArgsConstructor
public class PayoutService {

    private final PayoutRepository payoutRepository;
    private final SettlementBatchRepository batchRepository;
    private final MerchantServiceClient merchantClient;

    public PayoutResponseDTO createPayout(Long settlementId) {
        SettlementBatch batch = batchRepository.findById(settlementId)
                .orElseThrow(() -> new ResourceNotFoundException("Batch not found"));

        if ("PAID".equals(batch.getStatus())) {
            throw new IllegalStateException("Batch already paid");
        }

        // Get merchant's bank account ref via Feign
        String bankRef = fetchBankRef(batch.getMerchantId());

        Payout payout = new Payout();
        payout.setSettleBatchId(settlementId);
        payout.setMerchantId(batch.getMerchantId());
        payout.setAmount(batch.getNetAmount());
        payout.setBankAccountRef(bankRef);
        payout.setStatus("PENDING");
        payout.setPayoutDate(LocalDateTime.now());

        Payout saved = payoutRepository.save(payout);

        // Update batch status
        batch.setStatus("PAID");
        batchRepository.save(batch);

        // In real system: send to bank's NEFT/RTGS API here
        // For learning: just mark COMPLETED
        saved.setStatus("COMPLETED");
        payoutRepository.save(saved);

        return toResponse(saved);
    }

    @Async
    public CompletableFuture<PayoutResponseDTO> createPayoutAsync(Long settlementId)
```

```
        return CompletableFuture.completedFuture(createPayout(settlementId));
    }
}
```

8.1 Sync vs async payout

Two endpoints:

- `POST /payout/{settlementId}` — synchronous, caller waits for completion
- `POST /payout/async/{settlementId}` — returns immediately, processes in background

When you'd use async:

- Bulk payouts (1000+ merchants in one trigger)
- Don't want to keep an HTTP connection open for minutes
- Caller polls or listens for events

For `@Async` to work, application class needs `@EnableAsync` and a TaskExecutor bean configured.

8.2 Why fetch bank ref via Feign

The merchant's `bankAccountRef` is on `SettlementProfile` in merchant-service. Settlement-service doesn't store it locally because:

- It can change (merchant updates bank details)
- Always fetching ensures latest data
- Single source of truth

8.3 In-real-system note

```java
// In real system: send to bank's NEFT/RTGS API here
// For learning: just mark COMPLETED
```

In production, this is where you'd:

- Format an NEFT/RTGS payment message
- Call the bank's API
- Wait for acknowledgment

- Set status COMPLETED only on confirmation
- Handle failures (retry, alert ops)

For AcquirerX learning purposes, it's a fire-and-forget mark-as-completed.

---

9. Adjustment Handling

java

```java
public AdjustmentResponseDTO postAdjustment(AdjustmentRequestDTO dto) {
    Adjustment adj = new Adjustment();
    adj.setMerchantId(dto.getMerchantId());
    adj.setTxnId(dto.getTxnId());        // optional
    adj.setAmount(dto.getAmount());      // can be negative
    adj.setReason(dto.getReason());
    adj.setType(dto.getType());           // CHARGEBACK / REVERSAL / etc.
    adj.setStatus("POSTED");
    adj.setPostedDate(LocalDateTime.now());
    return toResponse(adjustmentRepository.save(adj));
}

public PagedResponseDTO<AdjustmentResponseDTO> getMerchantAdjustments(Long merchantI
```

How adjustments factor in next settlement: When the scheduler creates the next batch for a merchant, it should:

- Sum unsettled adjustments
- Subtract from netAmount
- Mark adjustments as factored-in

(Whether the current implementation does this fully is implementation-dependent — typically you'd track `settledIn` field on Adjustment.)

---

10. Feign Clients

10.1 MerchantServiceClient

java

```java
@FeignClient(name = "MERCHANT-SERVICE", fallback = MerchantServiceClientFallback.cla
public interface MerchantServiceClient {

    @GetMapping("/api/v1/merchants/{id}")
    Map<String, Object> getMerchantById(@PathVariable Long id);

    @GetMapping("/api/v1/merchants/status/{status}")
    Map<String, Object> getMerchantsByStatus(@PathVariable String status,
                                             @RequestParam int page,
                                             @RequestParam int size);

    @GetMapping("/api/v1/merchants/onboarding/settlement-profile/merchant/{merchantI
    Map<String, Object> getSettlementProfile(@PathVariable Long merchantId);
}
```

Or specifically:

```java
@GetMapping("/api/v1/merchants/status/ACTIVE")
Map<String, Object> getActiveMerchants(@RequestParam int page, @RequestParam int siz
```

10.2 TransactionServiceClient

```java
@FeignClient(name = "TRANSACTION-SERVICE", fallback = TransactionServiceClientFallba
public interface TransactionServiceClient {

    @GetMapping("/api/v1/txns/merchant/{merchantId}/unsettled")
    Map<String, Object> getUnsettledTxns(@PathVariable Long merchantId);

    @PutMapping("/api/v1/txns/merchant/{merchantId}/mark-settled")
    Map<String, Object> markTxnsSettled(@PathVariable Long merchantId);
}
```

10.3 Fallback strategies

```java
```

```java
@Component
public class TransactionServiceClientFallback implements TransactionServiceClient {

    @Override
    public Map<String, Object> getUnsettledTxns(Long merchantId) {
        // Return empty so settle() fails gracefully with "no txns"
        return Map.of("data", List.of());
    }

    @Override
    public Map<String, Object> markTxnsSettled(Long merchantId) {
        // Throw — if we can't mark settled, we shouldn't have created the batch
        throw new IllegalStateException("Transaction service unavailable - cannot ma
    }
}
```

Why these are different:

- Read fallback returns empty (settle just skips this merchant)
- Write fallback throws (we don't want orphan batches)

---

11. Controllers

11.1 SettlementController endpoints

```
GET    /settlement                                (paged)
POST   /settlement/search                         (paged with filter)
POST   /settlement/merchant/{merchantId}          (run settlement now for one
merchant)
GET    /settlement/merchant/{merchantId}          (paged history for a merchant)
GET    /settlement/{settleBatchId}/payouts        (payouts for a batch)
POST   /settlement/payout/{settleBatchId}         (create payout for batch)
POST   /settlement/adjustments                    (post adjustment)
GET    /settlement/adjustments/merchant/{merchantId}  (paged)
GET    /settlement/summary/merchant/{merchantId}
GET    /settlement/stats
```

11.2 PayoutController endpoints

```
POST   /payout/{settlementId}        (sync)
POST   /payout/async/{settlementId}  (async)
```

11.3 Why two controllers

- `SettlementController` is heavy — many endpoints
- Payouts are a sub-domain that warrants its own controller
- Keeps each controller under ~10 endpoints (good rule of thumb)

---

12. Repositories

java

```java
public interface SettlementBatchRepository extends JpaRepository<SettlementBatch, Lo
    Page<SettlementBatch> findByMerchantId(Long merchantId, Pageable pageable);
    long countByStatus(String status);

    @Query("SELECT s FROM SettlementBatch s WHERE " +
           "(:status IS NULL OR s.status = :status) AND " +
           "(:merchantId IS NULL OR s.merchantId = :merchantId) AND " +
           "(:minNet IS NULL OR s.netAmount >= :minNet) AND " +
           "(:maxNet IS NULL OR s.netAmount <= :maxNet) AND " +
           "(:from IS NULL OR s.postedDate >= :from) AND " +
           "(:to IS NULL OR s.postedDate <= :to)")
    Page<SettlementBatch> findByFiltersPaged(...);

    @Query("SELECT SUM(s.netAmount) FROM SettlementBatch s WHERE s.merchantId = :mer
    Double sumNetByMerchant(@Param("merchantId") Long merchantId);
}

public interface PayoutRepository extends JpaRepository<Payout, Long> {
    List<Payout> findBySettleBatchId(Long batchId);
    Page<Payout> findByMerchantId(Long merchantId, Pageable pageable);
}

public interface AdjustmentRepository extends JpaRepository<Adjustment, Long> {
    Page<Adjustment> findByMerchantId(Long merchantId, Pageable pageable);
}
```

---

13. application.properties

properties

```
server.port=9095
spring.application.name=settlement-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.datasource.url=jdbc:mysql://localhost:3306/acquirerx_settlement?createDatabas
spring.datasource.username=root
spring.datasource.password=kail3114
spring.jpa.hibernate.ddl-auto=update

# Feign + Resilience4J
spring.cloud.openfeign.circuitbreaker.enabled=true
resilience4j.circuitbreaker.instances.default.sliding-window-size=10
resilience4j.circuitbreaker.instances.default.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.default.wait-duration-in-open-state=30s
resilience4j.timelimiter.instances.default.timeout-duration=10s
# Longer timeout for settlement — aggregate queries can be slow

# Scheduler
spring.task.scheduling.pool.size=2

server.servlet.context-path=/api/v1
```

Note: timelimiter is 10s here vs 3s in transaction-service. Settlement aggregations are slow operations; 3s would frequently timeout.

---

14. The Full EOD Flow

```
1. Clock hits 1:00 AM
   ↓
2. SettlementScheduler.runEodSettlement() fires
   ↓
3. Feign → merchant-service: GET /merchants/status/ACTIVE?page=0&size=1000
   → 50 active merchants returned
   ↓
4. For each merchant (loop):
   a. SettlementService.settle(merchantId)
      - Feign → transaction-service: GET /txns/merchant/{id}/unsettled
        → Returns list of unsettled Txns
      - If empty → throw IllegalStateException → caught → skip++
```

```
      - Otherwise:
        - Aggregate: gross, fees, net, count, periodStart, periodEnd
        - Save SettlementBatch with status=POSTED
        - Feign → transaction-service: PUT /txns/merchant/{id}/mark-settled
        - success++
   ↓
5. Job logs: "success=42, skipped=8, failed=0"
   ↓
6. Next morning (or immediately):
   Operations runs payouts:
   POST /payout/{settlementId} for each POSTED batch
   - Fetches merchant's bankAccountRef via Feign
   - Creates Payout record, status=PENDING → COMPLETED
   - Updates batch status to PAID
```

---

15. Why This Design Is Robust

- **Scheduled but also manually triggerable** — recoverable if scheduler crashes
- **Per-merchant isolation** — one merchant's failure doesn't stop others
- **Circuit breakers** — merchant or transaction service downtime doesn't kill the scheduler
- **Status state machine** — batches go OPEN → POSTED → PAID, no skips
- **Skip vs fail accounting** — operations get clean signal vs noise
- **@Transactional** on settle — local atomicity
- **Sync + async payouts** — flexibility for different operational scenarios
- **Single source of truth** — bank account ref always fetched fresh from merchant-service

---

16. Honest Design Limits

Limit	Note
Distributed transaction risk	Cross-service "create batch + mark settled" not fully ACID; relies on idempotent retries
Payout integration	Stub — doesn't actually call any bank API
Adjustments not auto-applied	Posted but not deducted from next settlement automatically

Limit	Note
No saga pattern	Compensating transactions not built
Settlement window	Just "all unsettled" — no "settle daily for last 24h" cycle differentiation

These are documented limits suitable for a learning project. Real production settlement systems have far more complexity (reconciliation, reserve handling, multi-currency, holiday calendars).

---

17. Common Pitfalls

Pitfall	Symptom	Fix
`@EnableScheduling` missing	Scheduler never fires	Add to application class
Settlement runs twice for same period	Duplicate batches	Idempotency check on `markTxnsSettled`
Cron format wrong	Job runs at unexpected time	Use 6-part cron (Spring) not 5-part (Unix)
Bank ref hard-coded in batch	Stale on merchant bank change	Always fetch fresh via Feign
Negative netAmount possible	Adjustments larger than gross	Validate before payout
Async payout exceptions swallowed	Silent failures	Use `CompletableFuture.exceptionally`

---

18. Summary

Aspect	Detail
Purpose	EOD settlement, payouts, adjustments
Port	9095
DB	`acquirerx_settlement` (3 tables)

Aspect	Detail
Tech	Spring Boot, JPA, @Scheduled, Feign, Resilience4J
Services	`SettlementService`, `SettlementScheduler`, `PayoutService`
Feign clients	merchant, transaction (both with fallbacks)
Cron	`0 0 1 * * *` (1 AM daily)
Status flow	Batch: OPEN → POSTED → PAID
Payout modes	Sync + Async
Critical concern	Distributed-txn safety on settle + mark-settled

Next: Ops Service — disputes, reconciliation, notifications.